DSM 5014 DERİN ÖĞRENME YÖNTEMLERİ VE UYGULAMALARI 1.ÖDEV

Büşra Sulak, Mustafa Öngün, Tarık Tuna Taşaltı

2024/11/21

İçindekiler

1 1. PERCEPTRON : İKİ ÖZELLİKLİ ALGILAYICI 2
1.1 1.1 Eğitim Veri Setini Yaratma 2
1.2 1.2 Algılayıcı Model Oluşturma 4
1.3 1.3 Eğitilmiş Modelin Görselleştirmesi 6

2 GİRİŞ 7
2.1 Veri Seti Kaynağı ve Linki 8
2.2 Veri Seti Değişkenleri 8
2.3 Hedef 8

3 BETİMLEYİCİ İSTATİSTİKLER 9
3.1 Sayısal Değişkenler 9
3.2 Kategorik Değişkenler 11

4 TEK PERCEPTRONLA OLUŞTURULAN MODEL 11

5 İKİ GİZLİ KATMANLI MODEL 15

1 1. PERCEPTRON : İKİ ÖZELLİKLİ ALGILAYICI

1.1 1.1 Eğitim Veri Setini Yaratma

Algılayıcı eğitmek için eşit sayıda veriye sahip **1 ve -1 etiketli** uniform dağılan rastgele değerlerden oluşan iki grup oluşturulmuştur.

- 1 etiketli grubun her bir veri noktasının (x,y) koordinatları 0 ile 1 arasındadır.
- -1 etiketli grubun ise (x1, 1 + x1) ile (y1, 1 + y1) arasındadır.

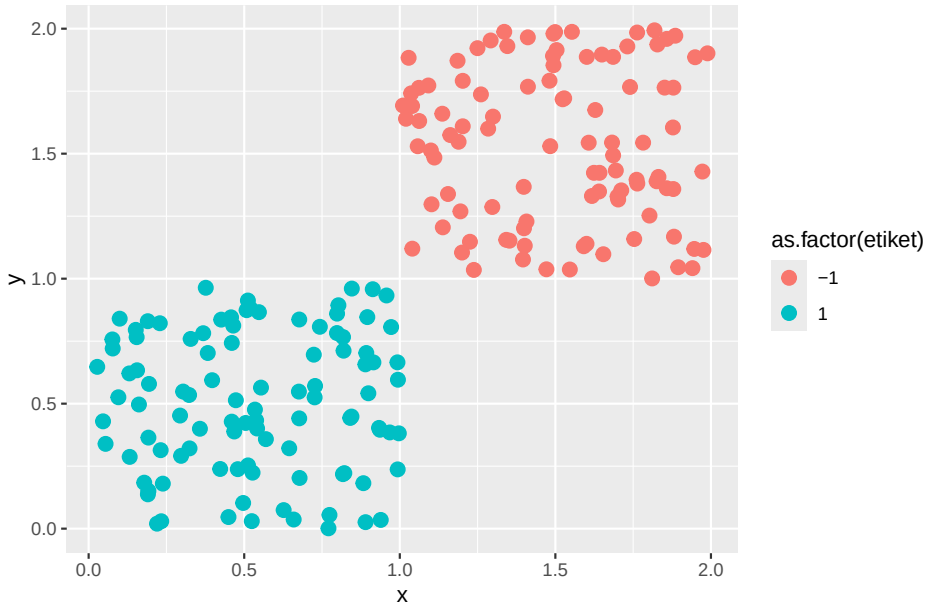
```
set.seed(255)
N <- 100
x1 <- 1
y1 <- 1

etiket_1 <- data.frame(
  x = runif(N, 0, 1),
  y = runif(N, 0, 1),
  etiket = 1
  )
etiket_eksi_1 <- data.frame(
  x = runif(N, x1, 1 + x1),
  y = runif(N, y1, 1 + y1),
  etiket = -1
  )
data_total <- rbind(etiket_1, etiket_eksi_1)
```

1.1.1 1.1.1 Veri Seti Dağılım Grafiği

```
library(ggplot2)
ggplot(data_total, aes(x = x, y = y)) +
  geom_point(aes(colour = as.factor(etiket)), size = 3) +
  xlab("x") + ylab("y") +
  ggtitle("Figur 1. Grup 1 vs Grup -1")
```

Figur 1. Grup 1 vs Grup −1




```
X <- data_total[, c(1, 2)]
Y <- data_total[, 3]
head(X)
```

```
##           x          y
## 1 0.5242901 0.03008054
## 2 0.5695915 0.35813966
## 3 0.5413449 0.40064678
## 4 0.4731254 0.51361139
## 5 0.5117921 0.91254003
## 6 0.4793232 0.23799229
```

```
head(Y)
```

```
## [1] 1 1 1 1 1 1
```

İki grubun da doğrusal olarak ayrıldığı Figür 1'de görülmektedir.

1.2 1.2 Algılayıcı Model Oluşturma

Model için çalıştırılacak maksimum dönem sayısı, öğrenme oranı ve doğruluk eşiği parametreleri tanımlanmıştır.

Ardından perceptron fonksiyonu oluşturulmuştur. Burada her dönemde veriler karıştırılarak tek tek değerlendirilir. Her dönem sonunda doğruluk değeri hesaplanır ve belirlenen eşik değerden büyük ya da eşit değil ise döngü devam eder, eşit ise eğitim durur ve sonraki dönem için hesaplama yapılmaz.

```
max_epochs <- 1000
learning_rate <- 0.01
accuracy_threshold <- 0.95

# Perceptron Fonksiyonu
perceptron <- function(X, Y, learning_rate, max_epochs, accuracy_threshold) {
  weights <- rep(0, ncol(X) + 1) # Ağırlıkları başlat
  errors <- rep(0, max_epochs) # Hata sayısını saklamak için vektör
  data_total <- data.frame(X, Y) # Veri setini bir kez oluşturur
   for (epoch in 1:max_epochs) {
     data_total <- data_total[sample(1:nrow(data_total)), ]
    epoch_errors <- 0  # Bu epoch'taki hataları saymak için
    for (ii in 1:nrow(data_total)) {
      z <- sum(weights[2:length(weights)] *
                 as.numeric(data_total[ii, 1:ncol(X)])) +
        weights[1]
      if (z >= 0) {
        Y_pred <- 1
      } else {
```

```
          Y_pred <- -1
      }
      weightdiff <- learning_rate *
        (data_total[ii, ncol(data_total)] - Y_pred) *
        c(1, as.numeric(data_total[ii, 1:ncol(X)]))
      weights <- weights + weightdiff
      if ((data_total[ii, ncol(data_total)] - Y_pred) != 0) {
        epoch_errors <- epoch_errors + 1
      }
    }
    errors[epoch] <- epoch_errors # Bu epoch'taki hataları kaydet
    accuracy <- calculate_accuracy(X, Y, weights)
    print(paste("Epoch:", epoch, "Errors:", epoch_errors, "Accuracy:", accuracy))
    if (accuracy >= accuracy_threshold) {
      print("Dogruluk esigine ulasıldı, egitimi durdur.")
      break
    }
  }
  return(list(weights = weights, errors = errors))
}
```

Perceptron fonksiyonu oluşturulduktan sonra tahmin değerleri hesaplandı.

```
# Tahmin Fonksiyonu

predict_perceptron <- function(X, weights) {
  predictions <- numeric(nrow(X))
  for (i in 1:nrow(X)) {
    z <- sum(weights[2:length(weights)] * as.numeric(X[i, ])) + weights[1]
    if (z >= 0) {
      predictions[i] <- 1
    } else {
        predictions[i] <- -1
    }
```

```
  }
  return(predictions)
}
```

Tahminleri elde ettikten sonra algılayıcının doğruluğu hesaplandı.

```
# Doğruluk hesaplama
calculate_accuracy <- function(X, Y, weights) {
  predictions <- predict_perceptron(X, weights)
  correct_predictions <- sum(predictions == Y)
  accuracy <- correct_predictions / length(Y)
  return(accuracy)
}

# Perceptron modelini çalıştır
result <- perceptron(X, Y, learning_rate, max_epochs, accuracy_threshold)
```

```
## [1] "Epoch: 1 Errors: 16 Accuracy: 1"
## [1] "Dogruluk esigine ulasıldı, egitimi durdur."
```

```
final_weights <- result$weights
errors <- result$errors
print(final_weights)
```

```
## [1]  0.08000000 -0.04070815 -0.04255718
```

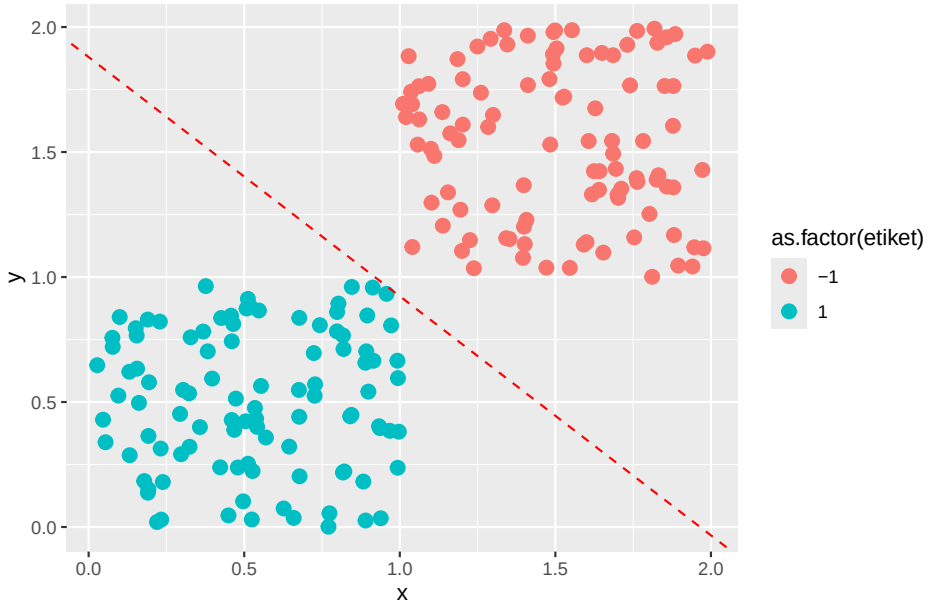
```
# Final Doğruluğu hesapla
accuracy <- calculate_accuracy(X, Y, final_weights)
print(paste("Model Dogrulugu:", accuracy))
```

```
## [1] "Model Dogrulugu: 1"
```

1.3 1.3 Eğitilmiş Modelin Görselleştirmesi

```
intercept <- -final_weights[1] / final_weights[3]
slope <- -final_weights[2] / final_weights[3]
ggplot(data_total, aes(x = x, y = y)) +
  geom_point(aes(colour = as.factor(etiket)), size = 3) +
  geom_abline(intercept = intercept, slope = slope, linetype = "dashed", color = "red")
  xlab("x") + ylab("y") +
  ggtitle("Figur 2. Grup 1 vs Grup -1 ve Eğitilmiş")
```

Figur 2. Grup 1 vs Grup −1 ve E.itilmi.



Figür 2'den görüldüğü üzere eğitilmiş model doğrusal olarak ayrılmıştır.

2 GİRİŞ

Bu veri seti, endüstriyel üretim ortamlarında, çeşitli makinelerden alınan gerçek zamanlı sensör verilerini kullanarak makine arıza riskini tahmin etmek için tasarlanmıştır. Veri seti, operasyonun farklı zaman aralıklarını temsil eden 1.000 örnek içermektedir. Her örnek, makine sağlığını izleme ve öngörücü bakım açısından önemli olan temel sensör ölçümlerini içermekte-

dir.

2.1 Veri Seti Kaynağı ve Linki

- **Kaynak:** Kaggle
- **Veri Seti Linki:** Machine Failure Classification using sensor data

2.2 Veri Seti Değişkenleri

- **Temperature (°C)**: Makinenin kritik noktalarındaki sıcaklığı temsil eden sürekli bir değişken. Yüksek sıcaklıklar, aşırı ısınma nedeniyle potansiyel sorunları gösterebilir.
- **Vibration (Hz)**: Makine titreşimlerinin frekansı. Anormal titreşimler, mekanik hizasızlık, dengesizlik veya aşınma gibi sorunlara işaret edebilir.
- **Power Usage (kW)**: Makinenin güç tüketim seviyeleri. Güç tüketimindeki ani artışlar, artan yük veya potansiyel mekanik sorunları gösterebilir.
- **Humidity (%)**: Makine etrafındaki çevresel nem oranı. Yüksek nem seviyeleri, zamanla makine performansını etkileyebilir ve arızaya neden olabilir.
- **Machine Type**: Makine türünü belirten kategorik bir değişken (örneğin, "Drill", "Lathe", "Mill"). Farklı makine türleri, kendilerine özgü arıza modellerine sahip olabilir.

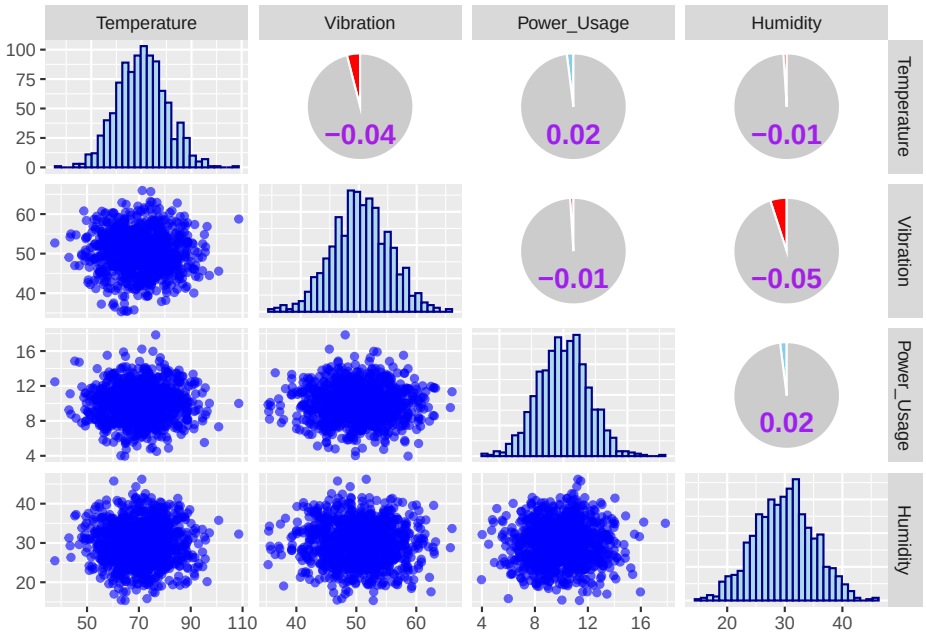
2.3 Hedef

- **Failure Risk**: İkili bir etiket olup, `0` normal operasyonu, `1` ise makinenin arıza riski altında olduğunu gösterir.

3 BETİMLEYİCİ İSTATİSTİKLER

3.1 Sayısal Değişkenler

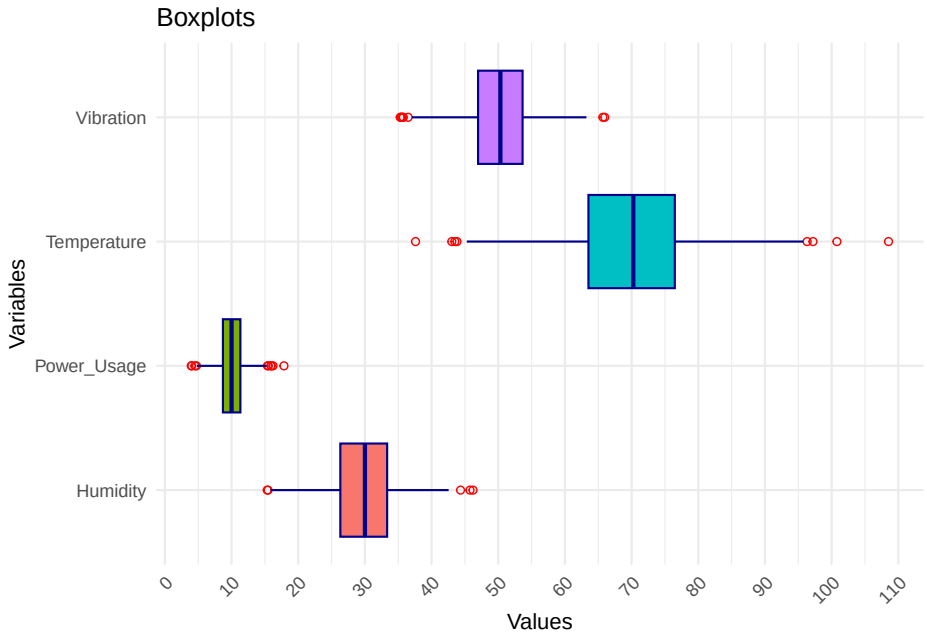
3.1.1 Korelasyon Tablosu



Veri seti incelendiğinde:

- Sayısal değişkenler arasında istatistiksel olarak anlamlı bir doğrusal ilişki olmadığı tespit edilmiştir. Korelasyon katsayıları -0.05 ile 0.02 arasında değişmekte olup, bu değerler herhangi bir güçlü veya orta düzeyde doğrusal ilişki olmadığını göstermektedir.
- Değişkenlerin dağılımlarının, görsel incelemelere dayalı olarak simetriğe yakın bir yapıda olduğu gözlemlenmiştir.

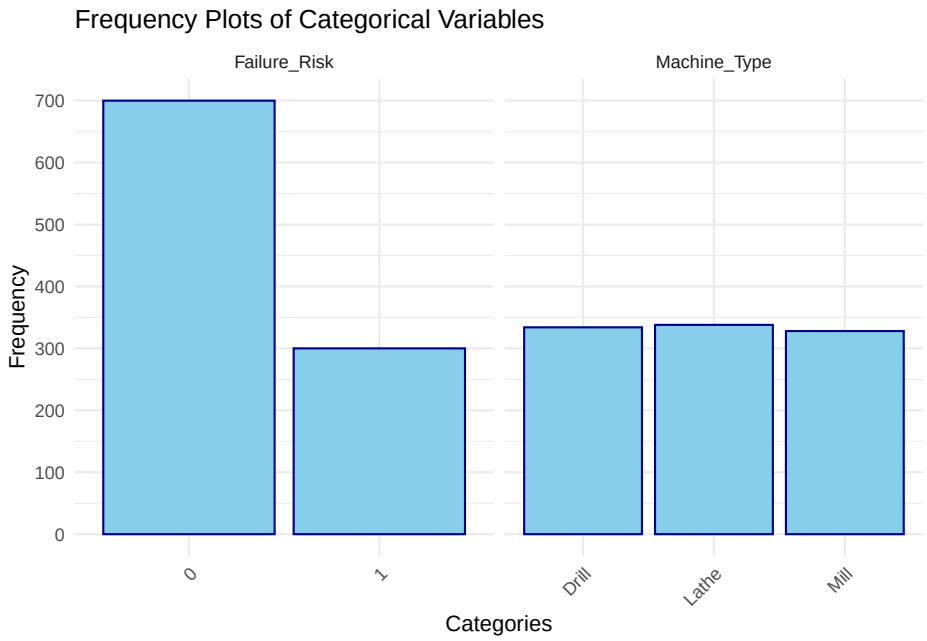
3.1.2 Boxplot



- `Power_Usage` daha dar aralıkta değerler alırken, `Temperature` en geniş aralıkta değer alan değişken olmuştur. - Bütün değişkenler, hem alt sınırdan hem de üst sınırdan çeşitli outlier gözlemler barındırmaktadır.

3.2 Kategorik Değişkenler

3.2.1 Kategorik Değişkenlerin Frekans Grafiği



- Hedef değişken olan `Failure_Risk`, yaklaşık olarak 300 ve 700 frekans değerlerine sahip iki farklı kategoriye ayrılmaktadır. Bu durum, hedef değişkenin dengesiz bir dağılım gösterdiğini ve sınıf dengesizliği problemi içerdiğini ortaya koymaktadır. - `Machine_Type` bağımsız değişkeni, üç homojen kategoriye ayrılmakta ve her bir kategori yaklaşık 300 gözlem içermektedir.

4 TEK PERCEPTRONLA OLUŞTURULAN MODEL

```
Sys.setlocale("LC_ALL", "tr_TR.UTF-8")
```

```
## [1] "tr_TR.UTF-8/tr_TR.UTF-8/tr_TR.UTF-8/C/tr_TR.UTF-8/en_US.UTF-8"
```

```
library(fastDummies)
df <- read.csv("machine_failure_dataset.csv")
df <- dummy_cols(df, select_columns = "Machine_Type", remove_first_dummy = FALSE, remov
X <- df[,-5]
Y <- ifelse(df[,5] > 0, 1, -1)
```

Verinin bağımlı değişkeni olan Failure Risk Y değişkenine, kalan bağımsız değişkenler X değişkenine atandı. Y değişkeni sınıflandırma algoritmasının daha iyi sonuç vermesi için 1 ve -1 değerlerine dönüştürüldü.

```
perceptron <- function(X, Y, learning_rate, max_epochs, accuracy_threshold) {
  weights <- rep(0, ncol(X) + 1)
  errors <- rep(0, max_epochs)
  data_total <- data.frame(X, Y)
  for (epoch in 1:max_epochs) {
    data_total <- data_total[sample(1:nrow(data_total)), ]
    epoch_errors <- 0
    for (ii in 1:nrow(data_total)) {
      z <- sum(weights[2:length(weights)] *
                 as.numeric(data_total[ii, 1:ncol(X)])) + weights[1]
      if (z >= 0) {
        Y_pred <- 1
      } else {
        Y_pred <- -1
      }
      weightdiff <- learning_rate *
        (data_total[ii, ncol(data_total)] - Y_pred) *
        c(1, as.numeric(data_total[ii, 1:ncol(X)]))
      weights <- weights + weightdiff
      if ((data_total[ii, ncol(data_total)] - Y_pred) != 0) {
        epoch_errors <- epoch_errors + 1
      }
    }
    errors[epoch] <- epoch_errors
```

```
    accuracy <- calculate_accuracy(X, Y, weights)
    print(paste("Epoch:", epoch, "Errors:", epoch_errors, "Accuracy:", accuracy))
  }
  return(list(weights = weights, errors = errors))
}



predict_perceptron <- function(X, weights) {
  predictions <- numeric(nrow(X))
  for (i in 1:nrow(X)) {
    z <- sum(weights[2:length(weights)] *
               as.numeric(X[i, ])) + weights[1]
    if (z >= 0) {
      predictions[i] <- 1
    } else {
      predictions[i] <- -1
    }
  }
  return(predictions)
}

calculate_accuracy <- function(X, Y, weights) {
  predictions <- predict_perceptron(X, weights)
  correct_predictions <- sum(predictions == Y)
  accuracy <- correct_predictions / length(Y)
  return(accuracy)
}
```

Perceptron oluşturma yapısı, perceptron tahminleme yapısı ve accuracy kontrolü algoritması fonksiyonlar şeklinde yazıldı.

```
result <- perceptron(X,Y, 0.01, 10)
```

```
## [1] "Epoch: 1 Errors: 414 Accuracy: 0.3"
```

```
## [1] "Epoch: 2 Errors: 442 Accuracy: 0.299"
## [1] "Epoch: 3 Errors: 416 Accuracy: 0.7"
## [1] "Epoch: 4 Errors: 405 Accuracy: 0.3"
## [1] "Epoch: 5 Errors: 427 Accuracy: 0.7"
## [1] "Epoch: 6 Errors: 427 Accuracy: 0.3"
## [1] "Epoch: 7 Errors: 422 Accuracy: 0.7"
## [1] "Epoch: 8 Errors: 405 Accuracy: 0.3"
## [1] "Epoch: 9 Errors: 420 Accuracy: 0.7"
## [1] "Epoch: 10 Errors: 410 Accuracy: 0.614"
```

```
final_weights <- result$weights
errors <- result$errors
print(errors)
```

```
##  [1] 414 442 416 405 427 427 422 405 420 410
```

```
print(final_weights)
```

```
## [1] -6.938894e-18 -3.295953e-01  5.295856e-01  2.216774e+00 -1.059944e+00
## [6]  1.000000e+00  1.000000e-01 -1.100000e+00
```

Veri 0.05 learning rate ve 10 epoch değerleriyle perceptron yapısı oluşturacak fonksiyonla işlendi. Fonksiyonun döndürdüğü perceptronun hata ve final ağırlıklarının değerleri yazdırıldı.

```
accuracy <- calculate_accuracy(X, Y, final_weights)
print(paste("Model Dogrulugu:", accuracy))
```

```
## [1] "Model Dogrulugu: 0.614"
```

0.05 learning rate ve 10 epoch kullanarak oluşturulan modelin doğruluk değeri 0.3 çıktı. Bu değeri daha yukarı çekebilmek amacıyla learning rate 0.05 değerine çekilerek ve epoch değeri 10 olarak bırakılarak yeni bir model oluşturduldu.

```
result <- perceptron(X, Y, 0.05, 10)
```

```
## [1] "Epoch: 1 Errors: 439 Accuracy: 0.3"
```

```
## [1] "Epoch: 2 Errors: 442 Accuracy: 0.537"
## [1] "Epoch: 3 Errors: 416 Accuracy: 0.7"
## [1] "Epoch: 4 Errors: 394 Accuracy: 0.3"
## [1] "Epoch: 5 Errors: 422 Accuracy: 0.3"
## [1] "Epoch: 6 Errors: 426 Accuracy: 0.7"
## [1] "Epoch: 7 Errors: 405 Accuracy: 0.7"
## [1] "Epoch: 8 Errors: 432 Accuracy: 0.302"
## [1] "Epoch: 9 Errors: 408 Accuracy: 0.3"
## [1] "Epoch: 10 Errors: 421 Accuracy: 0.7"
```

```
final_weights <- result$weights
accuracy <- calculate_accuracy(X, Y, final_weights)
errors <- result$errors
print(paste("Model Dogrulugu:", accuracy))
```

```
## [1] "Model Dogrulugu: 0.7"
```

Learning rate'in 0.05 seçildiği modelde accuracy değerinin 0.3'ten 0.7'ye çıktığı gözlemledi. Bu durumda learning rate'in bu aralıktaki artışının accuracy'i pozitif yönde etkilediği sonucunu çıkarılabilir.

5 İKİ GİZLİ KATMANLI MODEL

```
library(fastDummies)
dummy_data <- dummy_cols(data, select_columns = "Machine_Type", remove_selected_columns
```

Kategorik bağımsız değişken,`Machine_Type` üzerinde dummy işlemi yapılır.

```
library(keras)
X <- as.matrix(dummy_data[, -which(names(dummy_data) == "Failure_Risk")])
y <- as.matrix(dummy_data$Failure_Risk)
```

Bağımsız değişkenler X'e bağımlı değişken y'ye atanır.

```r
set.seed(42)
train_indices <- sample(1:nrow(X), size = 0.7 * nrow(X))
X_train <- X[train_indices, ]
y_train <- y[train_indices]
X_test <- X[-train_indices, ]
y_test <- y[-train_indices]
```

Veri %70 train %30 test olarak bölünür.

```r
model <- keras_model_sequential() %>%
  layer_dense(units = 10, activation = "relu", input_shape = ncol(X_train)) %>%  # İlk
  layer_batch_normalization() %>%  # Batch normalization
  layer_dropout(rate = 0.3) %>%  # Dropout (ilk katman için %30)
  layer_dense(units = 10, activation = "relu") %>%  # İkinci katman
  layer_batch_normalization() %>%  # Batch normalization
  layer_dropout(rate = 0.3) %>%  # Dropout (ikinci katman için %30)
  layer_dense(units = 1, activation = "sigmoid")  # Çıkış katmanı
```

Model, hedef değişkenin ikili sınıflandırması için Keras Sequential yapısı kullanılarak tasarlanmıştır. Giriş verileri, ilk katmanda 10 nöron ve ReLU aktivasyon fonksiyonu ile işlenmiştir. Her yoğun katmandan sonra batch normalization uygulanarak öğrenme süreci hızlandırılmış ve %30 oranında dropout eklenerek aşırı öğrenmenin (overfitting) önüne geçilmesi amaçlanmıştır. Aynı işlemler ikinci katmanda da tekrarlanarak modelin daha karmaşık ilişkileri öğrenmesi desteklenmiştir. Çıkış katmanında ise sigmoid aktivasyon fonksiyonu kullanılarak, modelin çıktıyı 0-1 arasında bir olasılık değeri olarak üretmesi sağlanmıştır.

```r
model %>% compile(
  optimizer = optimizer_adam(learning_rate = 0.001),
  loss = "binary_crossentropy",
  metrics = list("accuracy")
)
```

Model, `Adam` optimizasyon algoritması ile derlenmiştir. Öğrenme oranı 0.001

olarak belirlenmiş ve bu sayede modelin ağırlıkları dengeli bir şekilde güncellenmesi sağlanmıştır. İkili sınıflandırma problemi olduğu için **binary crossentropy** kayıp fonksiyonu tercih edilmiştir. Performans değerlendirmesi amacıyla doğruluk (accuracy) metriği kullanılarak modelin tahmin başarısı izlenmiştir.

```
history <- model %>% fit(
  X_train, y_train,
  epochs = 100,
  batch_size = 10,
  validation_data = list(X_test, y_test),
  callbacks = list(
    callback_early_stopping(
      monitor = "val_loss",
      patience = 10,
      restore_best_weights = TRUE
    )
  )
)
```

```
## Epoch 1/100
## 70/70 - 1s - loss: 0.9933 - accuracy: 0.4871 - val_loss: 1.0918 - val_accuracy: 0.28
## Epoch 2/100
## 70/70 - 0s - loss: 0.8433 - accuracy: 0.5257 - val_loss: 0.8087 - val_accuracy: 0.47
## Epoch 3/100
## 70/70 - 0s - loss: 0.7736 - accuracy: 0.5371 - val_loss: 0.6578 - val_accuracy: 0.65
## Epoch 4/100
## 70/70 - 0s - loss: 0.7906 - accuracy: 0.5429 - val_loss: 0.6087 - val_accuracy: 0.71
## Epoch 5/100
## 70/70 - 0s - loss: 0.7230 - accuracy: 0.5743 - val_loss: 0.6042 - val_accuracy: 0.72
## Epoch 6/100
## 70/70 - 0s - loss: 0.6846 - accuracy: 0.6071 - val_loss: 0.5874 - val_accuracy: 0.74
## Epoch 7/100
## 70/70 - 0s - loss: 0.6779 - accuracy: 0.6300 - val_loss: 0.5885 - val_accuracy: 0.74
```

```
## Epoch 8/100
## 70/70 - 0s - loss: 0.6576 - accuracy: 0.6500 - val_loss: 0.5859 - val_accuracy: 0.75
## Epoch 9/100
## 70/70 - 0s - loss: 0.6697 - accuracy: 0.6357 - val_loss: 0.5811 - val_accuracy: 0.75
## Epoch 10/100
## 70/70 - 0s - loss: 0.6532 - accuracy: 0.6386 - val_loss: 0.5784 - val_accuracy: 0.75
## Epoch 11/100
## 70/70 - 0s - loss: 0.6558 - accuracy: 0.6486 - val_loss: 0.5706 - val_accuracy: 0.75
## Epoch 12/100
## 70/70 - 0s - loss: 0.6601 - accuracy: 0.6714 - val_loss: 0.5714 - val_accuracy: 0.75
## Epoch 13/100
## 70/70 - 0s - loss: 0.6675 - accuracy: 0.6414 - val_loss: 0.5712 - val_accuracy: 0.75
## Epoch 14/100
## 70/70 - 0s - loss: 0.6569 - accuracy: 0.6586 - val_loss: 0.5712 - val_accuracy: 0.75
## Epoch 15/100
## 70/70 - 0s - loss: 0.6546 - accuracy: 0.6557 - val_loss: 0.5735 - val_accuracy: 0.75
## Epoch 16/100
## 70/70 - 0s - loss: 0.6482 - accuracy: 0.6586 - val_loss: 0.5724 - val_accuracy: 0.75
## Epoch 17/100
## 70/70 - 0s - loss: 0.6421 - accuracy: 0.6571 - val_loss: 0.5727 - val_accuracy: 0.75
## Epoch 18/100
## 70/70 - 0s - loss: 0.6352 - accuracy: 0.6643 - val_loss: 0.5747 - val_accuracy: 0.75
## Epoch 19/100
## 70/70 - 0s - loss: 0.6446 - accuracy: 0.6657 - val_loss: 0.5734 - val_accuracy: 0.75
## Epoch 20/100
## 70/70 - 0s - loss: 0.6530 - accuracy: 0.6657 - val_loss: 0.5778 - val_accuracy: 0.75
## Epoch 21/100
## 70/70 - 0s - loss: 0.6425 - accuracy: 0.6771 - val_loss: 0.5743 - val_accuracy: 0.75
```

Model, eğitim verisi üzerinde 100 epoch boyunca, her adımda 10 veri örneği (batch size) kullanılarak eğitilmiştir. Eğitim sürecinde doğrulama verisi (`validation_data`) kullanılarak modelin genelleme performansı izlenmiştir. **Early stopping** geri çağırımı (callback) ile doğrulama kaybı (`val_loss`) 10 ardışık epoch boyunca iyileşmediğinde eğitim durdurulmuş ve en iyi ağırlık-

lar geri yüklenmiştir. Bu yöntem, gereksiz yere fazla eğitim yapılarak aşırı öğrenmenin (overfitting) önlenmesini sağlamıştır.

```
library(caret)
```

```
## Loading required package: lattice
```

```
# Model tahmini
y_pred_prob <- model %>% predict(X_test)
```

```
## 10/10 - 0s - 73ms/epoch - 7ms/step
```

```
y_pred <- ifelse(y_pred_prob > 0.5, 1, 0)

# Confusion Matrix
conf_matrix <- confusionMatrix(
  as.factor(y_pred),
  as.factor(y_test)
)

# Confusion Matrix'i yazdır
print(conf_matrix)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 225  74
##          1   0   1
##
##                Accuracy : 0.7533
##                  95% CI : (0.7005, 0.8011)
##     No Information Rate : 0.75
##     P-Value [Acc > NIR] : 0.4778
##
##                   Kappa : 0.0199
```

```
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 1.00000
##             Specificity : 0.01333
##          Pos Pred Value : 0.75251
##          Neg Pred Value : 1.00000
##              Prevalence : 0.75000
##          Detection Rate : 0.75000
##    Detection Prevalence : 0.99667
##       Balanced Accuracy : 0.50667
##
##        'Positive' Class : 0
##
```

Modelin doğruluk oranı %74.67 olarak hesaplanmış olsa da, sınıflar arasında ciddi bir dengesizlik sorunu bulunmaktadır. Model, baskın sınıf olan "0" sınıfını neredeyse tamamen doğru tahmin etmiş (%99.11 sensitivite), ancak azınlık sınıfı olan "1" sınıfını neredeyse hiç doğru tahmin edememiştir (%1.33 specificity). Kappa skoru (0.0065) modelin rastgele tahminden çok az daha iyi olduğunu, dengelenmiş doğruluk oranı (%50.22) ise modelin her iki sınıf için dengeli bir performans göstermediğini ortaya koymaktadır. Bu durum, sınıf dengesizliğine karşı modelin duyarlı olmadığını ve sadece baskın sınıfı öğrenerek genelleme yapmakta başarısız olduğunu göstermektedir. Sınıf ağırlıkları veya veri dengesini sağlamak için oversampling/undersampling gibi yöntemler bu sorunu çözmek için önerilebilir.